
Calibrating and validating the loop against real fleet telemetry

The simulator has one feedback cycle: work → occupancy → power → temperature → throttle → clock → next interval's work. Each arrow is one equation with a few constants, and the occupancy in it is the **active** occupancy the rank actually retires over a sample window, not the static value its mesh size allows. This deck says which instrument on a real fleet measures each arrow, how its constants are fitted, and what must hold in real data before any fault label is trusted.

5

ARROWS IN THE
LOOP

14

CONSTANTS TO
FIT

3

INSTRUMENTS: DCGM, NSIGHT, THE
JOB

0

LABELLED INCIDENTS NEEDED FOR STAGES
1-2

Every arrow in the loop has a real instrument

Fleet-wide signals come from DCGM at about 1 Hz and are always on. Per-kernel truth comes from Nsight and is a one-off lab measurement per application. Timing comes from the job itself.

Arrow	Model term & constants	Real signal	Scope
work → active occupancy	$occ_{static} = cells / (cells + c\%), c\% = 250 \text{ k}$ $occ_{active} = cf \cdot occ_{static} + (1 - cf) \cdot 0.05$ $cf = \text{compute share of the window}$	static: Nsight Compute achieved occupancy & SM throughput per kernel. Active: <code>DCGM_FI_PROF_PIPE_FP64_ACTIVE</code> with <code>SM_ACTIVE</code> ; cf from NVTX phase timing	LAB static fleet active, 1 Hz
occupancy → power	$P = P_{idle} \cdot leak + (P_{max} - P_{idle}) \cdot occ \cdot f^{2.2}$ 70 W, 700 W, exponent 2.2	<code>DCGM_FI_DEV_POWER_USAGE</code> , <code>ENFORCED_POWER_LIMIT</code> ; clocks locked with <code>nvidia-smi -lgc</code>	fleet + one clock sweep
power → temperature	$dT/dt = (P \cdot R_{th} - (T - T_{inlet})) / \tau$ $R_{th} \text{ } 0.06 \text{ } ^\circ\text{C/W}, \tau \text{ } 20 \text{ s}, 0.9 \text{ } ^\circ\text{C/kW}$	<code>DCGM_FI_DEV_GPU_TEMP</code> ; inlet from the rack BMC or cold-aisle sensor; rack power from the PDU	fleet + one step test
temperature → throttle	engage at 84 °C, HW at 90 °C, 3 °C hysteresis, 0.020 clock/°C	<code>DCGM_FI_DEV_CLOCK_THROTTLE_REASONS</code> bitmask; <code>SLOWDOWN_TEMP</code> , <code>SHUTDOWN_TEMP</code> read from the device	fleet read-only
throttle → clock	1755 MHz base, 1050 min, quantised to 15 MHz steps	<code>DCGM_FI_DEV_SM_CLOCK</code> ; lattice from <code>nvidia-smi -q -d SUPPORTED_CLOCKS</code>	fleet read-only
clock → next work	compute $\propto 1 / f$, next interval; 88.7 ns per cell	the solver's per-rank phase timing (NVTX ranges under Nsight Systems, or its log)	job per iteration

`DCGM_FI_DEV_GPU_UTIL` is deliberately absent: it is the "utilisation" pinned at 99.9 % in every scenario, because a resident spin kernel counts as busy. Nothing in the loop is fitted to it.

Where does "occupancy" come from on a real GPU?

Two numbers share the name. **Static occupancy** is a kernel property set by subdomain size. **Active occupancy**, the simulator's `sm_occupancy_pct`, is that value time-weighted over the window by the share spent computing rather than spinning at the barrier. The fleet only sees the active one.

fleet DCGM profiling metrics

Per-GPU counters at ~1 Hz, ~1 % overhead, always on. `SM_ACTIVE`: cycles with a warp resident. `SM_OCCUPANCY`: resident warps / max. `PIPE_FP64/FP32/TENSOR_ACTIVE`: cycles the pipe retired work.

Role: measures **active** occupancy in production; what the detector runs on.

LAB Nsight Compute (ncu)

Per-kernel replay with full counters; tens of × overhead, so once per application on one node. `sm_warps_active` (achieved occupancy), `sm_throughput`, `dram_throughput`.

Role: fits **static** occupancy against subdomain size, and says which pipe carries the work (FP64 for this solver).

job Nsight Systems (nsys) + NVTX

Low-overhead timeline of kernels and NCCL / MPI calls; NVTX ranges tag `HALO`, `COMPUTE`, `ALLREDUCE`, `OUTPUT`.

Role: the real `rank_performance`: per-rank phase times, hence the compute share `cf` that turns static into active. The only way to see the barrier.

Composition rule: static from the lab, active from the fleet

$occ_{active} = cf \cdot occ_{static} + (1 - cf) \cdot occ_{spin}$: `cf` from NVTX phase timing, `occstatic` from Nsight Compute at that subdomain size, `occspin` what a rank retires while parked at the barrier. On the fleet: `occactive` ≈ `mean(PIPE_FP64_ACTIVE)`, `busy` ≈ `mean(SM_ACTIVE)`, `spinning` = `busy` - `occactive`. A parked rank shows `SM_ACTIVE` near 1 and `PIPE_ACTIVE` near 0; `occspin` (0.05) is read off that gap on healthy iterations. The mesh-size curve never meets the fleet directly.

Fitting the arrows, one experiment each

Every experiment is a sweep a fleet can run on one node in an afternoon. Each fits the constants of one arrow, so a bad fit points at one equation.

Arrow	Experiment	Fit	Expected shape
work → static occupancy	Solver on one GPU at 6 subdomain sizes (30 k – 8 M cells); ncu on the compute kernels	$c^{1/2}$ by least squares on $n/(n + c^{1/2})$; seconds per cell update from kernel time / cells	saturation in subdomain size; independent of waiting
static → active occupancy	Same job at full scale; DCGM at 1 Hz plus NVTX phase timing per rank	occ_{spin} from PIPE_ACTIVE inside all-reduce ranges; predict $cf \cdot occ_{static} + (1 - cf) \cdot occ_{spin}$ per window vs measured PIPE_ACTIVE	tracks DCGM within a few %; residual flat in wait share
occupancy → power	Lock clocks (<code>nvidia-smi -lgc</code>) at 5 frequencies × 3 subdomain sizes, 60 s each; read <code>POWER_USAGE</code>	P_{idle} idle; exponent from the log–log slope of $(P - P_{idle})$ vs f at fixed occupancy; P_{max} at base clock, full occupancy	log–log slope near 2–2.5; linear in occupancy at fixed f
power → temperature	Step test idle → full load → idle, 5 min each; <code>GPU_TEMP</code> at 1 Hz, rack inlet sensor, rack kW from the PDU	τ from the exponential rise; $R_{th} = (T_{ss} - T_{inlet}) / P$; rack coupling from inlet vs rack kW over a day	one time constant fits rise and fall; residual white
temperature → throttle	No sweep: read <code>SLOWDOWN_TEMP</code> , <code>SHUTDOWN_TEMP</code> ; mine <code>CLOCK_THROTTLE_REASONS</code> transitions	threshold directly; hysteresis from engage–release pairs; derate per °C from clock vs temperature while throttled	engage clusters at one temperature; release a few °C lower
throttle → clock	Read <code>SUPPORTED_CLOCKS</code> ; every throttled second in history	step and floor directly; every observed <code>SM_CLOCK</code> on the lattice	a staircase, never a ramp
the lag	Same step test, 1 Hz	lagged cross-correlation of the threshold crossing vs the first clock drop	peak at +1 sample: the governor acts next interval

Seven things that must be true of real telemetry before any label exists

These test the shape of the loop, not its numbers. Each is a scripted check over ordinary healthy production days; a failure names the missing physics rather than a constant to tune.

#	Claim the loop makes	Check on fleet data	Pass looks like
1	Power follows work, not busy time	Regress <code>POWER_USAGE</code> on <code>PIPE_ACTIVE</code> and separately on <code>GPU_UTIL</code>	R^2 far higher for pipe-active; the <code>GPU_UTIL</code> fit is flat
2	Temperature is first-order in power	Fit the step-test τ to every natural load ramp in history (job starts)	one τ across GPUs, no second time constant
3	Inlet is a rack property	PCA of the 32 GPU temperatures in a rack after removing each GPU's own power term	first component carries most variance and tracks the inlet sensor
4	Throttling is a quantised staircase with hysteresis	Histogram of <code>SM_CLOCK</code> during throttled seconds; engage vs release temperatures	mass on the 15 MHz lattice; release 2–4 °C below engage
5	The governor acts one interval late	Lagged correlation of threshold crossings vs clock drops	peak at +1 sample, not 0
6	Active occupancy is a time-weighted mix	Per window, $cf \cdot occ_{static} + (1 - cf) \cdot occ_{spin}$ from NVTX timing vs measured <code>PIPE_ACTIVE</code> ; slope of <code>PIPE_ACTIVE</code> against <code>cf</code> across ranks	slope $\approx occ_{static} - occ_{spin}$; intercept $\approx occ_{spin}$; no dependence on the mesh beyond occ_{static}
7	The barrier identity	From NVTX timing: per rank, halo + compute + wait + output = iteration time; wait of fast ranks = slowest rank's arrival – own arrival	identity holds to the ms; utilisation stays ~100 % while pipe-active drops on waiting ranks

Only after 1–7 pass is a scenario worth replaying. If 1 fails the power arrow needs a bandwidth term; if 3 fails the thermal domain is not the rack; if 6 fails waiting ranks do not spin as modelled and the util-versus-occupancy gap is no signature; if 7 fails the job is not bulk-synchronous.

Replay the scenarios on the fleet, controlled first, then historical

Each simulated scenario has a safe real counterpart that can be staged on a drained node or rack. Score the simulator's telemetry and the diagnosis against what the fleet recorded.

Scenario	Controlled real counterpart	What to compare
thermal	CRAC set-point raised on one rack during a maintenance window, or a blanked cold-aisle tile	inlet ramp, 32-GPU co-movement, time to first throttle, staircase depth
gpu degradation	<code>nvidia-smi -lgc</code> cap at 90 % on one GPU of a running job	occupancy drop on the victim, rank spread, throttle-reason bit, peers' spin signature
network domain	Administratively down 7 of 8 uplinks on one leaf; keep one	queue depth, drops, halo time on cross-rack ranks only, compute unchanged
straggler	A CPU-side process pinned to one host stealing cycles from the launch thread	intermittent pacing duty, near-zero wait on the victim, no throttle bit
phase change	Turn checkpoint interval from 100 to 20 iterations in the job's own config	storage latency stays up, io pressure, rank spread flat, no device channel moves

Then history: every ticketed incident with a known cause, replayed through the same 100-iteration windowing and scored the way the simulator is scored, verdict and localisation against the ticket.

What counts as passing

Not numbers to the percent: the order of events, which channels move and which stay flat, and the diagnosis reaching the ticket's verdict and rack or GPU.

What a failure means

A signature the fleet shows and the simulator does not is a missing model term, never a constant to re-tune after stage 1 has fitted it.

Boundary to state up front

The simulator stops at throttling. A real cooling failure ends in thermal shutdown and a manual restart; that state is not modelled and no replay is scored past it.

IF THE ASSUMPTIONS CHANGE

What moves in the loop under a different workload or a different failure

The cycle is the invariant. A new workload changes the constants and the pipe that stands in for work; a new failure changes where the fault enters the cycle.

Different workload

Assumption	What changes
ML training instead of CFD	work pipe becomes <code>PIPE_TENSOR_ACTIVE</code> ; occupancy curve refitted per model; all-reduce is bandwidth-bound (gradients, GB not bytes), so the barrier arrow gains a size term
Memory-bound solver	power arrow gains a <code>DRAM_ACTIVE</code> term; the clock exponent falls because HBM does not scale with SM clock
Asynchronous or pipelined communication	the barrier identity weakens; rank spread stops being the headline diagnostic, straggler signatures blur
Subset allocations, mixed tenants	fleet-relative features must be computed over the job's GPUs only; idle neighbours are the most common thing in the data

Different failure

Failure	Where it enters the loop
Power cap (facility or board)	between power and clock: <code>THROTTLE_REASONS</code> power bit, clock falls with temperature flat; a new throttle reason, same staircase
HBM ECC storm	at occupancy: row remapping steals bandwidth; like gpu degradation but with <code>XID</code> / ECC counters as the witness
NIC flap, single optic	outside the loop, at the halo: per-port asymmetry the current fabric model cannot show; first roadmap item
Shared-aisle cooling	thermal domain becomes several racks; check 3 two slides back is what detects it
Thermal shutdown	past the loop's boundary: an absorbing state that needs an availability model, not a telemetry one

The test for any new assumption is the same three stages: fit its constants from a sweep, check its structure on healthy days, replay its fault under control. The arrows that did not change keep their fits.